1. Instrukcja warunkowa if w Bashu

Instrukcja warunkowa if służy do weryfikacji, czy określony warunek jest prawdziwy. Jeśli wynik testu to TRUE, wykonywane są polecenia umieszczone po słowie kluczowym then. Całą konstrukcję obowiązkowo kończy słowo fi.

Podstawowa składnia (sam warunek if)

Bash

```
if warunek
then
	polecenie
fi
```[cite: 75]

Przykład: Skrypt sprawdzający obecność pliku konfiguracyjnego `.bashrc` w katalogu domowym (zapis `~/` oznacza katalog domowy użytkownika lub zmienną `$HOME`)[cite: 75]:

```bash
#!/bin/bash
if [ -e ~/.bashrc ]
then
	echo "Masz plik .bashrc"
fi
```[cite: 75]

Rozbudowa o gałąź alternatywną (`else`)

W sytuacji, gdy test warunku zakończy się wynikiem negatywnym (`FALSE`), można wykonać alternatywny zestaw poleceń umieszczony po słowie kluczowym `else`[cite: 75]:

```bash
if warunek
then
	polecenie1
else
	polecenie2
fi
```[cite: 76]

Przykład:

```bash
#!/bin/bash
if [ -e ~/.bashrc ]
then
	echo "Masz plik .bashrc"
else
	echo "Nie masz pliku .bashrc"
fi
```[cite: 76]

Testowanie wielu warunków (`elif`)
Jeśli pierwszy warunek nie jest spełniony, można sprawdzić kolejne warianty za pomocą słowa kluczowego `elif`[cite: 76]:

```bash
if warunek1
then
	polecenie1
elif warunek2
then
	polecenie2
else
	polecenie domyślne
fi
```[cite: 76]

Przykład (sprawdzanie lokalizacji środowiska KDE):
```bash
#!/bin/bash
if [ -x /opt/kde/bin/startkde ]
then
	echo "Masz KDE w katalogu /opt"
elif [ -x /usr/bin/startkde ]
then
	echo "Masz KDE w katalogu /usr"
elif [ -x /usr/local/bin/startkde ]
then
	echo "Masz KDE w katalogu /usr/local"
else
	echo "Nie wiem gdzie masz KDE"
fi
```[cite: 76]

---

2. Łączenie warunków i poleceń w listy (AND / OR)

W skryptach Bash można budować bardziej złożone warunki i struktury sterujące bez konieczności zagnieżdżania wielu instrukcji `if`, wykorzystując operatory logiczne.

Operator AND (`&&`) – iloczyn logiczny
Składnia[cite: 76]:
```bash
instrukcja1 && instrukcja2 && instrukcja3 && instrukcja4
```[cite: 76]

* **Zasada działania:** Instrukcje są wykonywane sekwencyjnie od lewej do prawej tak długo, dopóki wynik całości wyrażenia nie jest jednoznacznie określony[cite: 76].
* Jeśli `instrukcja1` zwróci kod błędu (`FALSE`, czyli różny od zera), kolejne instrukcje **nie zostaną wykonane**, ponieważ cały iloczyn logiczny przyjmie wartość fałszywą[cite: 76].
* Aby wyrażenie zwróciło sukces (`TRUE`, czyli kod powrotu `0`), **wszystkie** instrukcje muszą wykonać się poprawnie[cite: 76].

Operator OR (`||`) – suma logiczna
Składnia[cite: 77]:
```bash
instrukcja1 || instrukcja2 || instrukcja3 || instrukcja4
```[cite: 77]

* **Zasada działania:** Podobnie jak w przypadku AND, instrukcje są przetwarzane od lewej do prawej[cite: 77].
* Jeśli **dowolna** z instrukcji zakończy się sukcesem i przyjmie wartość `TRUE`, cały ciąg przerywa dalsze wykonywanie, ponieważ wartość całego wyrażenia jest już określona jako prawdziwa[cite: 77].
* Zwrócenie wartości `FALSE` przez daną instrukcję wymusza przejście do wyhodowania kolejnego elementu listy[cite: 77].

> ⚠ **Ostrzeżenie i wskazówka:**
> * Operatory `&&` oraz `||` mają taki sam priorytet i są przetwarzane ściśle od lewej do prawej[cite: 77].
> * Aby zmienić kolejność wyliczeń lub pogrupować bardziej skomplikowane warunki, należy stosowa